

RISKRADAR: AN EXPLAINABLE MACHINE LEARNING SYSTEM FOR WOMEN'S SAFETY RISK ASSESSMENT IN URBAN AREAS

A PROJECT BASED LEARNING (PBL) REPORT

Submitted by

N. VENKATESAN (2104251040604)

NEETHIVENDHAN T. (2104251040629)

Submitted in partial fulfilment of the requirements

for the

Project-Based Learning component of Machine Learning

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI



Affiliated to Anna University, Chennai
(Autonomous)

SEPTEMBER 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

- IM1.** To create next generation leaders by effective teaching learning methodologies and in still Scientifics Park in them to meet the global challenges.
- IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.
- IM3.** To inculcate human values and ethical principles to cater the societal needs.
- IM4.** To contribute towards the research ecosystem by providing a suitable,

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

- DM1.** To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.
- DM2.** To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.
- DM3.** To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

BONAFIDE CERTIFICATE

This is to certify that the Project–Based Learning report titled "RiskRadar: An Explainable Machine Learning System for Women's Safety Risk Assessment in Urban Areas" is a Bonafide record of work carried out by N. VENKATESAN (2104251040604) and NEETHIVENDHAN T. (2104251040629) of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor–guided Project-Based Learning (PBL) component of the Machine Learning course during the academic year 2026–2027 under my supervision.

SIGNATURE

Dr. S. Pavithra, M.E., Ph.D.
Professor and Head,
Dept. of Computer Science and
Engineering
Chennai Institute of Technology,
Chennai – 69.

SIGNATURE

Vignesh T.
MENTOR
Professor
Dept. of Computer Science and
Engineering
Chennai Institute of Technology,
Chennai – 69.

Submitted for the final review held on

Internal Examiner

DECLARATION

We jointly declare that the PBL report on "RiskRadar: An Explainable Machine Learning System for Women's Safety Risk Assessment in Urban Areas" is the result of original work done by us and best of our knowledge, similar work has not been submitted to "ANNA UNIVERSITY, CHENNAI" for the requirement of Degree of BACHELOR OF ENGINEERING. This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of COMPUTER SCIENCE AND ENGINEERING.

Signature

N. VENKATESAN (2104251040604)

NEETHIVENDHAN T. (2104251040629)

Place: Chennai

Date: 18.09.2026

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honorable Chairman **SHRI. P. SRIRAM** for providing immense facilities at our institution.

We are very proudly rendering our thanks to our Principal **Dr.A.RAMESH M.E, Ph.D.**, for the facilities and the encouragement given by him to the progress and completion of our project.

We would like to express special thanks of gratitude to our Dean **Dr. V. SRINIVASA RAO, M.E., Ph.D.**, who has been the key spring of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr. S. PAVITHRA M.E, Ph.D.**, for her effective leadership, encouragement and guidance in the project.

We would like to extend our thanks to the Project Co-ordinator «PROJECT CO-ORDINATOR - NAME AND DESIGNATION», Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from the class advisors Vignesh T., Professor, and Ayesha Nasreen, Assistant Professor, of the Department of Computer Science and Engineering and others for providing valuable suggestions and for the successful completion of the project.

N. VENKATESAN (2104251040604)
NEETHIVENDHAN T. (2104251040629)

ABSTRACT

Urban safety is neither uniform across a city nor constant through the day: the same street can be unremarkable at 2 p.m. and genuinely hazardous at 2 a.m. Most existing safety applications are reactive, mapping incidents only after they occur. This project develops RiskRadar, an explainable machine learning system that classifies any location-hour as Low, Medium or High risk for women's safety before an incident happens. A corpus of 100,000 area-hour records covering the Chennai metropolitan region, with 29 usable attributes spanning crime counts, street lighting, CCTV coverage, population density, footfall, emergency-service proximity and environmental conditions, was expanded into 79 criminologically-motivated features through a leakage-free, row-wise transform. Nine models were benchmarked on an identical stratified 80/20 split; a tuned LightGBM classifier achieved 98.27% accuracy, 97.68% macro F1 and 0.9993 ROC-AUC on 20,000 unseen records, confirmed by $98.00\% \pm 0.08$ three-fold cross-validation. Critically, the model produced zero Low-to-High confusions, never once labelling a dangerous area safe. Exact Shapley values explain every prediction, and a counterfactual scan identifies which realistic municipal intervention would lower an area's risk band.

Keywords: women's safety, explainable machine learning, gradient boosting, feature engineering, SHAP

TABLE OF CONTENTS

Right-click here and choose "Update Field" in Word to generate the table of contents with page numbers.

Team Roles and Responsibilities.....	i
CHAPTER 1: INTRODUCTION.....	1
1.1 Background.....	1
1.2 Driving Question.....	1
1.3 Objectives.....	1
1.4 Scope and Limitations.....	1
CHAPTER 2: CONCEPT EXPLORATION.....	1
2.1 Related Approaches.....	1
2.2 Summary Table.....	1
2.3 What This Told Us.....	1
CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION	1
3.1 Weekly PBL Progress Log.....	1
3.2 Requirements.....	1
3.3 Feasibility.....	1
CHAPTER 4: ITERATIVE DESIGN AND DEVELOPMENT.....	1
4.1 System Architecture.....	1
4.2 Iteration 1 — Baseline.....	1
4.3 Iteration 2 — Refinement.....	1
4.4 Final Approach.....	1
4.5 Training Procedure.....	1
CHAPTER 5: IMPLEMENTATION.....	1
5.1 Module Description.....	1
5.2 Key Code Snippets.....	1
5.3 User Interface / Demo (if applicable).....	1
CHAPTER 6: RESULTS AND DISCUSSION.....	1
6.1 Evaluation Metrics.....	1
6.2 Results Across Iterations.....	1
6.3 Discussion.....	1
6.4 Limitations.....	1
CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES	1

7.1 Individual Reflections.....	1
7.2 Team Learning.....	1
7.3 Course Outcomes — Evidence Summary.....	1
CHAPTER 8: CONCLUSION AND FUTURE SCOPE.....	1
8.1 Conclusion.....	1
8.2 Future Scope.....	1
REFERENCES.....	1
APPENDIX.....	1
A.3 Self and Peer Assessment.....	1

LIST OF FIGURES

Figure 1.1 Exploratory analysis — high-risk share by hour and by police distance	15
Figure 4.1 System architecture diagram	23
Figure 6.1 Confusion matrix and per-class performance of the deployed model	31
Figure 6.2 Model benchmark across all nine candidates	31
Figure 6.3 Top 15 features of the deployed model	32

LIST OF TABLES

Table 2.1 Summary of related approaches	18
Table 3.1 Weekly PBL progress log	20
Table 3.2 Hardware and software requirements	21
Table 6.1 Model evaluation results across iterations	30
Table A.1 Self and peer assessment	39

LIST OF ABBREVIATIONS

API — Application Programming Interface

AUC — Area Under the Curve

CCTV — Closed-Circuit Television

CV — Cross-Validation

ECE — Expected Calibration Error

GBM — Gradient Boosting Machine

LightGBM — Light Gradient Boosting Machine

MCC — Matthews Correlation Coefficient

ML — Machine Learning

OvR — One-vs-Rest

PBL — Project-Based Learning

REST — Representational State Transfer

ROC — Receiver Operating Characteristic

SHAP — SHapley Additive exPlanations

CHAPTER 1

INTRODUCTION

1.1 Background

Personal safety in public space is one of the most persistent constraints on how freely women move through a city. The constraint is not evenly distributed: it varies sharply by locality, by the quality of physical infrastructure such as street lighting and camera coverage, by how far help is, and above all by the time of day. A street that is busy, well-lit and overlooked at 6 p.m. can become isolated and unlit by midnight, and the risk it presents changes with it.

Almost every deployed safety application today is reactive. Incident-mapping platforms, emergency SOS buttons and crowd-sourced reporting tools all activate at or after the moment of harm. They are valuable, but they answer the question "what happened here?" rather than "how risky is this place right now?" For someone deciding which route to walk, or for a municipal authority deciding where the next twenty streetlights should go, the predictive question is the more useful one.

Machine learning is well suited to this because the inputs that plausibly drive environmental risk — recorded crime by category, working and broken streetlights, CCTV density, population density, pedestrian footfall, distance to the nearest police station and hospital, weather and visibility — are all measurable, and their interactions are non-linear. Broken streetlights barely matter at noon and matter a great deal at 2 a.m. A model that captures such interactions can convert routinely collected municipal data into an actionable risk estimate.

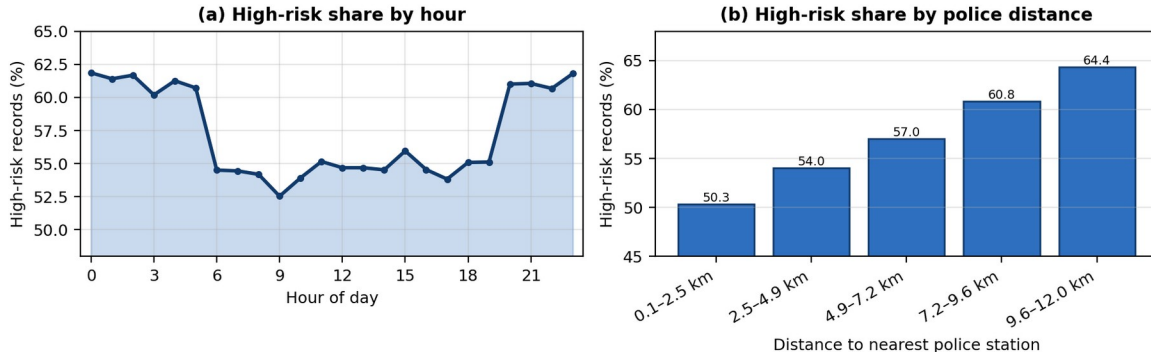


Figure 1.1 Exploratory analysis of the RiskRadar corpus. (a) The share of records labelled High risk is elevated overnight (61.2% mean between 20:00 and 05:00) against 54.8% between 10:00 and 16:00. (b) High-risk share rises monotonically with distance from the nearest police station, from 50.3% in the closest quintile to 64.4% in the furthest. Note the truncated vertical axes.

1.2 Driving Question

Driving question: "Can the safety risk level of an urban area at a given hour be reliably predicted from routinely measurable crime and infrastructure indicators — and how much of that prediction is actually driven by physical infrastructure that a city can change?"

The second half of that question is what made the project investigable rather than a fixed specification. It is not enough to output a risk label; the team wanted to know whether lighting,

camera coverage and police proximity carry measurable predictive weight, because only those factors are within a municipality's control.

Narrowed to a concrete machine learning task: given 29 measured attributes describing one area during one hour, classify that area-hour into one of three ordered risk bands — Low, Medium or High — and attribute each individual prediction back to the specific inputs that drove it, so that the verdict can be inspected and challenged rather than merely trusted.

1.3 Objectives

- To acquire, validate and preprocess the RiskRadar dataset of 100,000 area-hour records, establishing a clean, leakage-free basis for supervised classification.
- To design and iteratively refine a domain-driven feature-engineering transform that converts 29 raw attributes into criminologically-motivated predictive features.
- To build, benchmark and tune multiple classification models, promoting the best performer on evidence rather than on assumption.
- To evaluate model performance using accuracy, macro F1, ROC-AUC, calibration error and a confusion-matrix error analysis, and to justify every design choice against those results.
- To integrate explainability through exact Shapley values, so that each prediction is accompanied by the reasoning behind it, and to expose the model through a working service and interface.
- To document weekly progress and mentor feedback across the PBL cycle, and to reflect on the team's approach, division of work and individual skill growth.

1.4 Scope and Limitations

In scope. The project covers the complete supervised-learning pipeline for three-class risk classification: data validation, feature engineering, model benchmarking and tuning, evaluation, per-prediction explanation, a counterfactual intervention scan, a REST service and a browser-based dashboard. All 100,000 available records are used, split 80,000 for training and 20,000 as an untouched hold-out set.

Out of scope. The system does not perform real-time data ingestion from live municipal feeds, does not track or identify individuals, and is not deployed on public infrastructure; it runs locally as a self-hosted service. No mobile application was built.

Limitations that bound the claims. The corpus is synthetic, modelled on Chennai rather than drawn from verified municipal crime records, so the reported accuracy describes performance on that distribution and should be treated as optimistic for real deployment. Recorded crime is itself a biased proxy for actual crime, and under-reporting is most severe for harassment and

assault — precisely the offences this project cares about most. Geographic coverage is limited to Chennai coordinates; transfer to another city would require refitting and revalidation.

CHAPTER 2

CONCEPT EXPLORATION

2.1 Related Approaches

2.1.1 Theoretical grounding. Two ideas from criminology shaped the feature design more than any algorithm choice. Cohen and Felson's routine activity theory [1] holds that a crime requires the convergence of a motivated offender, a suitable target and the absence of a capable guardian — which implies that risk should be modelled as an interaction between opportunity and supervision, not as a raw incident count. Jacobs' concept of "eyes on the street" [2] argues that ordinary pedestrian presence is itself a form of informal surveillance. Both ideas were translated directly into engineered features.

2.1.2 Classical machine learning approaches. Kounadi et al. [3] systematically reviewed spatial crime forecasting and found tree-ensemble methods to be the most consistent performers on tabular municipal data, frequently outperforming more complex alternatives once features encode spatial and temporal context. Breiman's random forests [4] and the gradient-boosting family — XGBoost [5] and LightGBM [6] — dominate this space. LightGBM's histogram-based, leaf-wise growth makes it particularly efficient on datasets of this size and shape.

2.1.3 Deep learning and spatio-temporal approaches. Catlett et al. [7] applied spatio-temporal predictive models to smart-city crime data, and Bogomolov et al. [8] showed that behavioural and demographic signals from mobile data can forecast crime hotspots. These approaches achieve strong results but require large volumes of fine-grained sequential data, and their outputs are considerably harder to explain — a serious drawback for a safety application whose verdicts must be challengeable.

2.1.4 Explainability. Lundberg and Lee's unified SHAP framework [9] provides additive feature attributions with desirable theoretical properties, and has an exact, efficient solution for tree ensembles. Mitchell et al. [10] argue for model cards as a disclosure standard covering intended use, performance across groups and known limitations. The project adopted both.

2.2 Summary Table

Ref.	Approach / Model	Dataset	Reported Result
[1]	Routine activity theory (criminological framework)	Conceptual	Basis for the opportunity-vs- guardianship feature design
[3]	Systematic review of spatial crime	Multiple municipal datasets	Tree ensembles most

	forecasting		consistent on tabular data
[4]	Random Forest	Various benchmark datasets	Strong baseline; robust to feature scaling
[5]	XGBoost (gradient-boosted trees)	Various benchmark datasets	State-of-the-art on structured/tabular problems
[6]	LightGBM (histogram-based, leaf-wise boosting)	Large-scale tabular benchmarks	Comparable accuracy at substantially lower training cost
[7]	Spatio-temporal predictive models	Smart-city crime data	Effective, but needs fine-grained sequential data
[8]	Demographic + mobile behavioural data	Mobile network and census data	Crime hotspots predictable from behavioural signals
[9]	SHAP additive feature attribution	Model-agnostic	Exact, efficient attributions for tree ensembles

2.3 What This Told Us

Three conclusions came out of this exploration and directly set the plan for Iteration 1. First, on tabular data of this size a well-engineered gradient-boosted tree ensemble was the right starting point — not a neural network, which would demand more data than available and forfeit interpretability. Second, the literature consistently suggested that how features encode context matters more than model capacity, so the team committed to investing effort in feature engineering before reaching for a bigger model. Third, for a safety system explainability could not be an afterthought, so SHAP was designed into the architecture from the outset rather than bolted on at the end.

CHAPTER 3

PROJECT PLANNING AND TEAM ORGANISATION

3.1 Weekly PBL Progress Log

Week	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem framing and dataset study	Defined the driving question; audited the 100,000-record corpus (no missing values, no duplicates); confirmed the three-class target and its 2.90:1 imbalance.	Narrow the question to a buildable ML task before writing any code.
3–4	Concept exploration and baseline plan	Reviewed criminological theory and spatial crime-forecasting literature; selected tree ensembles as the starting family and committed to SHAP for explainability.	Justify the model family from the literature rather than by default.
5–7	Iteration 1 — baseline model	Built decision tree (91.92%) and random forest (94.45%) on raw label-encoded features; identified encoder-persistence, identifier-leakage and no-pipeline defects.	Accuracy is not the interesting problem yet — make it reproducible first.
8–10	Iteration 2 — feature engineering and refactor	Moved all preprocessing into a scikit-learn Pipeline; engineered 29 raw attributes into 79 features; logistic regression reached 97.47% and histogram gradient boosting 97.86%.	Quantify how much the features alone contribute — run a controlled ablation.
11–12	Final evaluation, service and report	Tuned LightGBM to 98.27% (CV 98.00% $\pm$ 0.08); added SHAP attribution and the counterfactual scan; built the REST API and dashboard; wrote 61 automated tests.	Report the error profile, not just the headline accuracy.

3.2 Requirements

Dataset. The RiskRadar dataset comprises 100,000 area-hour records described by 31 columns, covering the Chennai metropolitan region. Of these, 29 columns are used as model inputs, one (Risk_Level) is the target, and one (Area_ID) is deliberately discarded because it is unique to every row and would allow a tree to memorise individual records rather than learn generalisable structure. The data contains no missing values, no duplicate rows and no constant columns. The target has three ordered classes — High (57,296 records, 57.3%), Low (22,974, 23.0%) and Medium (19,730, 19.7%) — giving a modest class-imbalance ratio of 2.90:1. Inputs span five groups: crime counts by category, temporal attributes, lighting and surveillance infrastructure, population and footfall, and emergency-service proximity plus environmental conditions.

Category	Requirement
Processor / RAM	Intel Core i5 (or equivalent) / 8 GB RAM minimum; the final model trains in approximately 30 seconds on 80,000 records
Programming language	Python 3.10 or newer (developed and verified on Python 3.13)
Libraries / frameworks	scikit-learn, LightGBM, SHAP, pandas, NumPy, FastAPI, Pydantic, Uvicorn, joblib, matplotlib
Development environment	Visual Studio Code and Jupyter Notebook; the dashboard runs in any modern browser
Testing	pytest (61 automated backend tests) and a jsdom-based frontend smoke test
Version control	https://github.com/venkat200709

3.3 Feasibility

The project was achievable within the PBL timeframe for three reasons. The dataset was already available in a clean tabular form, removing the data-collection risk that typically consumes the first half of a student project. The entire toolchain — Python, scikit-learn, LightGBM, SHAP and FastAPI — is open-source and runs comfortably on a standard laptop; the final model trains in approximately 30 seconds on 80,000 records, which made the build-test-learn cycle fast enough to complete several genuine iterations rather than one. Finally, the weekly plan in Table 3.1 deliberately front-loaded problem framing and concept exploration, so that by the time implementation began the team had already decided what to build and why.

CHAPTER 4

ITERATIVE DESIGN AND DEVELOPMENT

4.1 System Architecture

Figure 4.1 shows the end-to-end pipeline. It divides into three stages — data preparation, modelling, and serving — described below.

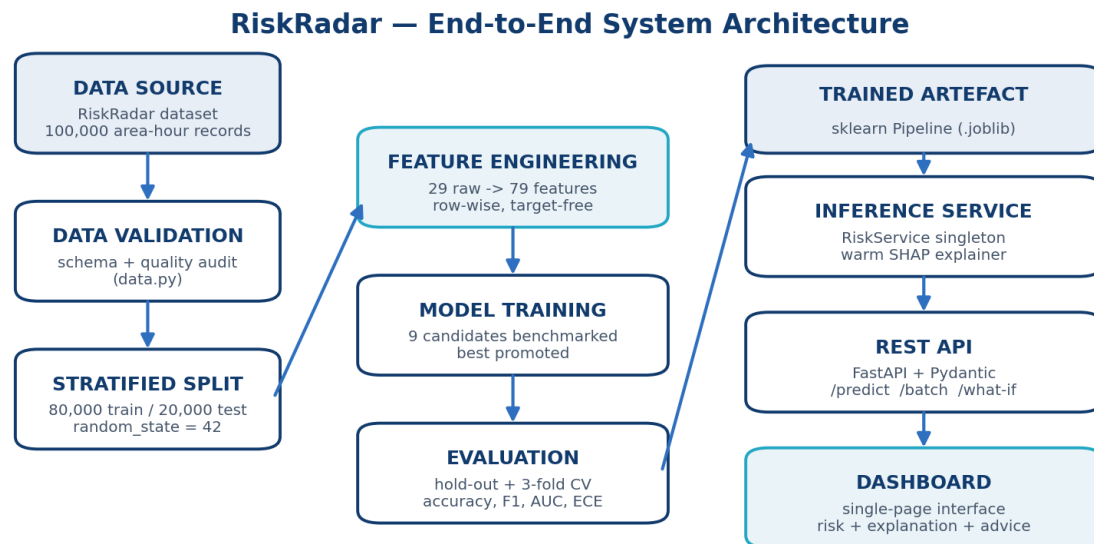
Data source and validation. The raw CSV is loaded and checked against an explicit schema contract: every required column must be present and the target may contain only the three expected class labels. A quality audit reports missing values, duplicates, constant columns and class balance, so that any defect is caught before it reaches a model.

Stratified split. The records are divided 80,000 / 20,000 with stratification on the target and a fixed random seed, so both halves preserve the class ratio and the split is exactly reproducible.

Feature engineering. The 29 raw attributes are expanded into 79 features by a purely row-wise transform. Because it is wrapped as the first step of the fitted pipeline, the identical code executes during training and during live inference — which structurally eliminates the most common cause of a model that scores well offline and misbehaves in production.

Model training and evaluation. Nine candidate models are trained on the same features and the same split, and the best performer is promoted automatically from the resulting leaderboard. The chosen model is assessed on the untouched hold-out set and by three-fold cross-validation on the training set.

Serving. The fitted pipeline is persisted as a single artefact, loaded once by an inference service that also holds a pre-built SHAP explainer, and exposed through a validated REST API to a single-page dashboard that presents the risk band, the confidence, the explanation and the recommended interventions.



The feature-engineering transform is step one of the fitted pipeline, so training and serving execute identical code.

Figure 4.1 End-to-end system architecture, from raw dataset through feature engineering and model selection to the served dashboard.

4.2 Iteration 1 — Baseline

Iteration 1 established the simplest thing that could work, in order to have an honest baseline to improve on. The 29 raw attributes were used almost as-is: the six categorical columns (Day, Weather, Visibility, Previous_Risk, Crime_Trend) were integer-encoded and everything else was passed through unchanged. A decision tree and a random forest were trained on the same stratified split used throughout the project.

Results. The decision tree reached 91.92% accuracy and the random forest 94.45%. An earlier gradient-boosting attempt on raw features reached 94.83%, which became the reference baseline for the rest of the project.

What the baseline revealed. Reviewing this iteration surfaced three defects that mattered more than the accuracy figure. First, a single label-encoder instance was being reused across every categorical column and was never persisted, so the encoding could not be reproduced at inference time. Second, Area_ID was still present in the feature matrix, inviting the model to memorise rows. Third, and most seriously, there was no pipeline: preprocessing lived in the training script and would have had to be re-implemented by hand in any application that consumed the model, which is exactly how training and serving drift apart.

Mentor feedback at this review was that the accuracy number was not the interesting problem — reproducibility was. That comment set the direction for Iteration 2.

4.3 Iteration 2 — Refinement

Iteration 2 changed the feature representation rather than the model, and restructured the code so that the result was reproducible.

Structural changes. All preprocessing was moved inside a scikit-learn Pipeline, with the feature transform as step one, so that a saved model consumes raw records directly. Area_ID was dropped. The arbitrary label encoding was replaced with hand-specified ordinal mappings that carry real meaning — Poor < Medium < Good for visibility, Decreasing < Stable < Increasing for crime trend — so the numeric ordering is informative rather than accidental.

Feature engineering. The 29 raw attributes were expanded to 79 features under three self-imposed rules: no feature may read the target; no feature may use cross-row statistics, so a single record can be scored in isolation and no train/test information can leak; and raw counts must be normalised by exposure, because 40 incidents means something very different in a dense commercial hub than in a quiet residential lane. Representative features include Darkness_Exposure (broken-light ratio $\times$ night flag $\times$ poor visibility, encoding that lamp failures only matter after dark), Surveillance_Deficit (weighted crime severity $\div$ combined CCTV and lighting coverage), Unpoliced_Crime_Load (crime severity $\times$ distance to police), Guardianship_Index (footfall $\div$ population density, operationalising "eyes on the street"), and cyclical hour encodings so that 23:00 is adjacent to 00:00 rather than maximally distant.

Results. On the engineered features, logistic regression — the simplest available classifier — reached 97.47%, and histogram gradient boosting reached 97.86%. A controlled ablation isolated the contribution: holding the model, the split and the hyperparameters fixed and changing only the representation, logistic regression moved from 93.89% on raw columns to 97.44% on engineered features, a gain of 3.55 percentage points. That single change contributed more than the entire subsequent move to tuned gradient boosting, which added a further 0.80 points.

4.4 Final Approach

The team converged on a tuned LightGBM classifier. LightGBM is a gradient-boosting decision-tree framework [6]: it builds an additive ensemble of shallow trees, where each new tree is fitted to the gradient of the loss function with respect to the current ensemble's predictions, so every tree corrects the residual errors of its predecessors. Two design features make it well matched to this problem — histogram-based split finding, which buckets continuous features and makes training on 80,000 records $\times$ 79 features a matter of seconds,

and leaf-wise growth, which expands the leaf offering the largest loss reduction rather than growing level by level.

Why it fits. The risk signal in this data proved to be smooth and largely additive rather than deeply interactive, which a boosted ensemble of narrow trees captures efficiently. The hyperparameter sweep confirmed this directly: widening the trees consistently hurt performance. Configurations with 255 and 127 leaves underperformed, while 31 leaves with more boosting rounds performed best — evidence of overfitting at higher capacity rather than of an under-powered model.

Final hyperparameters. `n_estimators = 1800`, `learning_rate = 0.05`, `num_leaves = 31`, `min_child_samples = 25`, `subsample = 0.85` with `subsample_freq = 1`, `colsample_bytree = 0.85`, `reg_lambda = 1.0`, `objective = multiclass` over three classes, `random_state = 42`.

A weighted soft-voting ensemble over the four strongest models was also constructed and evaluated. It scored 98.22%, marginally below the single tuned LightGBM at 98.27%, so the simpler model was deployed. Building it was nonetheless instructive: an initial near-uniform weighting across five members scored below its own best member, because averaging in the two weakest learners actively degraded the vote. Restricting membership to models within one accuracy point of the leader, and weighting by a temperature-scaled softmax over accuracy, recovered the loss. When candidates are separated by fractions of a point, a plain average is the wrong prior.

4.5 Training Procedure

Split. 80,000 training records and 20,000 hold-out test records, stratified on the target with `random_state = 42`. The hold-out set was not examined during feature design or hyperparameter tuning.

Cross-validation. Three-fold stratified cross-validation on the training set, used to confirm that the hold-out result was not an artefact of one fortunate split. The deployed model scored $98.00\% \pm 0.08$ across folds (98.11%, 97.97%, 97.93%), which agrees closely with the 98.27% hold-out figure.

Objective and evaluation. Multiclass log loss was the training objective. Model selection used hold-out accuracy with macro F1 as a tie-breaker; the promotion of the winning model is automated in the training script rather than chosen by hand.

Reproducibility. Every random seed is fixed, the split is deterministic, and the training run writes a metadata record capturing the library versions, the complete feature list and the full metric suite alongside the model artefact. An automated test re-scores the hold-out set from the saved artefact and fails if the reported accuracy cannot be reproduced.

CHAPTER 5

IMPLEMENTATION

5.1 Module Description

config — a single source of truth for file paths, the raw input schema, the ordinal mappings for categorical attributes, the class ordering and all training constants. Every other module imports from here, so no constant is defined twice.

data — dataset loading, schema validation and the stratified split. It raises an explicit error if a required column is missing or the target contains an unexpected label, and produces the data-quality audit reported in Section 3.2.

features — the 79-feature transform. It is the analytical core of the project and is constrained to be purely row-wise and target-free, properties enforced by unit tests.

models — the model registry. Each of the nine candidates is defined here as a complete pipeline, so the trainer, the tests and this report all describe identical configurations.

train — benchmarking, tuning, ensembling and evaluation. It trains every candidate on the same split, computes the full metric suite, promotes the winner and writes the model artefact together with its metadata, leaderboard and pre-aggregated analytics.

explain — the explainability layer. It builds a SHAP TreeExplainer over the deployed model and performs the counterfactual intervention scan that identifies which realistic change would lower an area's risk band.

service — the inference layer. A single service object loads the artefact and the explainer once at start-up and is reused for every request, so a prediction costs one forward pass rather than a disk read and deserialisation.

schemas and api — the REST interface. Pydantic models validate and range-check every field at the boundary, and FastAPI exposes prediction, batch scoring, sensitivity analysis, analytics and health endpoints with automatically generated OpenAPI documentation.

frontend — a single self-contained HTML dashboard presenting seven views: overview, area assessment, batch scoring, analytics, geospatial risk map, model card and project information.

5.2 Key Code Snippets

```
# ---- 1. Feature engineering: the darkness-exposure interaction -----
```

```

# Broken lamps are irrelevant at noon and dangerous at 2 a.m.
out["Is_Night"] = hour.isin(NIGHT_HOURS).astype(float)
out["Broken_Light_Ratio"] = out["Broken_Streetlights"] / (lights + EPS)
out["Darkness_Exposure"] = (out["Is_Night"]
                             * out["Broken_Light_Ratio"]
                             * (3.0 - out["Visibility_Score"]))

# --- 2. The pipeline: feature transform is step one -----
# Training and serving therefore execute identical code.
def make_pipeline(estimator):
    return Pipeline([
        ("features", FunctionTransformer(engineer, validate=False)),
        ("model", estimator),
    ])

# --- 3. The deployed model -----
model = make_pipeline(LGBMClassifier(
    n_estimators=1800, learning_rate=0.05, num_leaves=31,
    min_child_samples=25, subsample=0.85, subsample_freq=1,
    colsample_bytree=0.85, reg_lambda=1.0, random_state=42))
model.fit(X_train, y_train) # X_train holds RAW columns

# --- 4. Exact SHAP attribution for one prediction -----
explainer = shap.TreeExplainer(model.named_steps["model"])
shap_values = explainer.shap_values(engineer(record))

```

5.3 User Interface / Demo (if applicable)

The dashboard is served by the API at <http://127.0.0.1:8000> and presents the model through seven views. The Assess Area view takes the 29 inputs through grouped form controls, returns the predicted risk band with its confidence and a 0–100 safety score, and displays the SHAP attribution as a signed waterfall separating risk-increasing from protective factors. Below it, the intervention panel names the specific municipal action that would lower the risk band, and a sensitivity sweep traces how the safety score responds as a single input is varied across its range. Further views provide batch CSV scoring with a downloadable annotated file, exploratory analytics, a geospatial risk map and a full model card.

[Insert screenshot(s) of the running dashboard here — run the project locally, open <http://127.0.0.1:8000>, and capture the Overview and Assess Area views. Caption each as Figure 5.1, Figure 5.2 and so on.]

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Evaluation Metrics

Accuracy alone is a weak summary for this task. The dataset is moderately imbalanced (2.90:1 between the largest and smallest class), so a model could score well on accuracy while performing poorly on the smallest class. A set of complementary metrics was therefore used.

Accuracy and macro F1. Accuracy reports overall correctness; macro F1 averages the per-class F1 scores with equal weight, so under-performance on the smaller Medium class cannot be hidden by strength on the larger High class.

ROC-AUC (one-vs-rest) and log loss. These assess the quality of the predicted probabilities rather than only the final label, which matters because the interface surfaces a confidence value to the user.

Cohen's kappa and Matthews correlation. Both measure agreement corrected for the level expected by chance, giving a more conservative view than raw accuracy on imbalanced data.

Expected calibration error. ECE compares stated confidence against observed accuracy across ten confidence bins. It was included specifically because the dashboard displays confidence: an over-confident model would actively mislead a user.

Confusion-matrix error analysis. For a safety system the distribution of errors matters more than their count. Mistaking a High-risk area for Low-risk is a qualitatively different failure from mistaking it for Medium, so the confusion matrix was treated as a primary result rather than a supporting detail.

6.2 Results Across Iterations

Version	Accuracy	Precision	Recall / F1
Iteration 1 — Decision Tree (raw features)	91.92%	89.26%	89.09%
Iteration 1 — Random Forest (raw features)	94.45%	92.74%	92.64%
Reference baseline — earlier project version	94.83%	not reported	not reported
Iteration 2 — Logistic Regression (engineered)	97.47%	96.58%	96.56%
Iteration 2 — Hist Gradient Boosting	97.86%	97.13%	97.15%
Weighted soft-vote ensemble	98.22%	97.60%	97.62%
Final approach — LightGBM (tuned)	98.27%	97.65%	97.68%

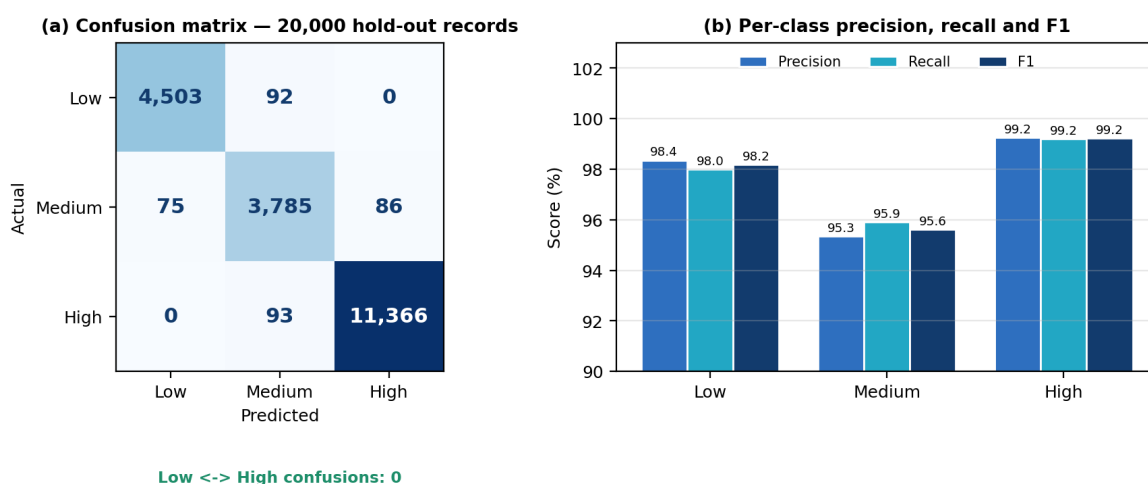


Figure 6.1 (a) Confusion matrix of the deployed model on the 20,000-record hold-out set; the two corner cells are exactly zero. (b) Per-class precision, recall and F1.

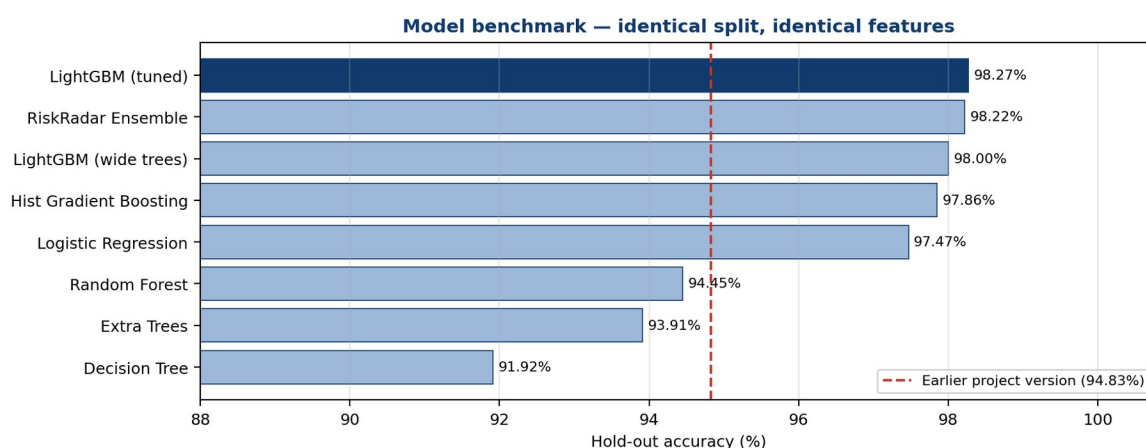


Figure 6.2 Hold-out accuracy of the eight trained candidates on an identical split with identical features. The dashed line marks the project's earlier baseline of 94.83%. The majority-class baseline (57.29%, Table 6.1) is omitted from this chart for scale.

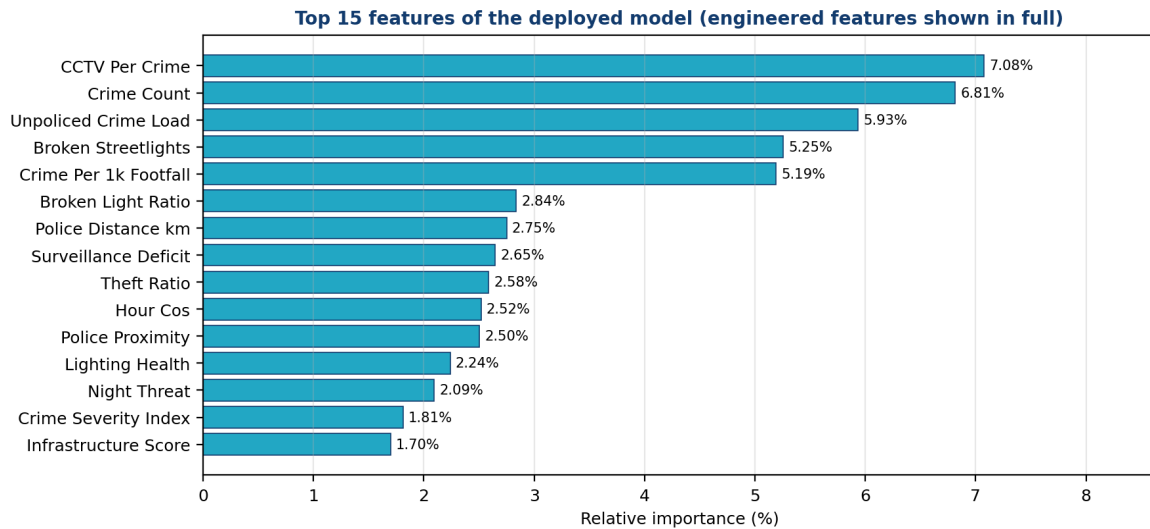


Figure 6.3 The fifteen strongest features of the deployed model. Seven of the top ten are engineered rather than raw, and the leading signals are ratios of threat to protection.

6.3 Discussion

Feature representation, not model capacity, drove the improvement. This is the clearest finding of the project. Logistic regression — a linear model with no capacity to represent interactions — reaches 97.47% on the engineered features, outperforming every tree model trained on the raw columns, including the random forest at 94.45%. The controlled ablation quantifies it: +3.55 points from representation alone, against +0.80 points from the entire subsequent upgrade to tuned gradient boosting. The engineered features had already made the problem close to linearly separable, and the boosting model was refining a structure that the features exposed.

The feature-importance profile confirms the model learned structure rather than noise. Seven of the top ten features are engineered rather than raw, and the strongest signals are ratios of threat to protection — CCTV cameras per incident (7.08%), crime severity weighted by distance from police (5.93%), crime per thousand pedestrians (5.19%) and the surveillance-deficit ratio (2.65%) — rather than absolute crime volume. This is what routine activity theory predicts: risk arises from the balance between opportunity and guardianship, not from incident counts alone. It also answers the second half of the driving question: infrastructure that a municipality can actually change carries substantial predictive weight.

The error profile is the right shape for a safety application. The two corner cells of the confusion matrix are exactly zero. Across 20,000 hold-out predictions the model never confused a High-risk area with a Low-risk one in either direction. All 346 errors are single-band slips adjacent

to Medium, which is both the smallest class and, by construction, the fuzziest boundary. The catastrophic failure mode for this system — telling a user that a dangerous street is safe — did not occur once.

Narrower trees generalised better. The hyperparameter sweep showed accuracy falling as tree width increased: 255 and 127 leaves both underperformed 31 leaves. Rather than treating this as a tuning detail, it is evidence about the data — the underlying signal is smooth and additive, so additional model capacity fits noise instead of structure.

Individually weak attributes became jointly strong once combined. Examined one at a time, several raw attributes carry almost no marginal signal: visibility and crime trend each separate the High-risk share by under one percentage point, and the relationship between working-streetlight ratio and risk is not even monotonic (55.9%, 61.8%, 57.9% and 53.1% across the four lighting bands). Yet lighting-derived features are among the model's strongest predictors. The explanation is that risk in this data lives in interactions rather than in marginal effects — broken lamps matter only after dark and only where visibility is poor, which is precisely what `Darkness_Exposure` encodes. This is the clearest evidence for why engineering the interactions explicitly outperformed handing the raw columns to a more powerful model.

Calibration is strong, which makes the displayed confidence meaningful. An expected calibration error of 0.0013 means a stated 90% confidence corresponds to roughly 90% empirical accuracy. Combined with cross-validation at $98.00\% \pm 0.08$ — a spread of less than a tenth of a point, closely matching the 98.27% hold-out result — this indicates the model is neither overfitted to the split nor over-confident in its outputs.

6.4 Limitations

The dataset is synthetic. This is the single most important qualification on every number in this report. The corpus is modelled on Chennai but is not verified municipal crime data, so the reported metrics describe performance on that synthetic distribution. Real performance would be lower and must be re-established on audited data before any deployment.

Reported crime is a biased proxy for actual crime. Under-reporting is systematic and is most severe for harassment and assault — the offences most central to this project. A model trained on reported incidents therefore learns the distribution of reporting as well as the distribution of harm.

Model selection introduces mild optimism. The winning model was chosen on hold-out accuracy, which makes that figure slightly optimistic as an estimate of generalisation. The cross-validated $98.00\% \pm 0.08$ is the more conservative number and is reported alongside it for that reason.

Medium is the weakest class. At 95.63% F1 against 98.18% for Low and 99.22% for High, the middle band absorbs almost all residual error. A three-band scheme is inherently coarse, and the boundaries between bands are the least well-defined part of the target.

No temporal drift handling. The model is fitted to a static snapshot. Crime patterns and infrastructure change, so a deployed version would need periodic retraining and a drift monitor, neither of which was in scope.

Feedback-loop risk. If patrol allocation were driven by these predictions, more crime would be recorded where patrols are sent, raising predicted risk there further. Any real deployment would need to monitor for this explicitly. Predicted risk should be treated as an argument for investing in an area, never for withdrawing services from it.

CHAPTER 7

TEAM REFLECTION AND LEARNING OUTCOMES

7.1 Individual Reflections

N. Venkatesan: «Describe the modules you personally owned — for example the feature-engineering transform and the model benchmarking — one thing you learned, and one challenge you faced. 2–4 sentences.»

Neethivendhan T.: «Describe the modules you personally owned — for example the FastAPI service, the dashboard and the test suite — one thing you learned, and one challenge you faced. 2–4 sentences.»

Suggested material to draw on: designing features from criminological theory rather than by trial and error; learning that a scikit-learn Pipeline is what keeps training and serving consistent; discovering through the ablation that representation mattered more than model choice; and finding that getting the project to run reliably on a different machine was a distinct problem from getting the model to work.

7.2 Team Learning

What worked. Splitting the work along module boundaries rather than by task type meant each member owned a coherent part of the system end to end and could be held responsible for it, while the shared configuration module kept the interfaces between those parts stable. Writing tests alongside the feature transform rather than afterwards repeatedly caught mistakes early — the constraint that features must be row-wise, for instance, is easy to state and easy to violate accidentally, and the test made a violation immediately visible.

What the mentor feedback changed. The most consequential review comment came after Iteration 1: that the accuracy figure was less interesting than whether it was reproducible. That redirected the whole of Iteration 2 away from chasing a higher number and towards restructuring the code into a pipeline, dropping the identifier column and replacing the unpersisted encoder. The accuracy improvement followed from that work rather than being pursued directly.

What we would do differently. Two things. First, we would write the ablation experiment earlier — we only quantified the contribution of feature engineering near the end, and having that number sooner would have justified spending even more time on features. Second, we

would test the deployed interface on a clean machine much earlier. Several issues that cost real time appeared only outside our development environment, and they were environment and packaging problems rather than model problems. Verifying that the model works is not the same as verifying that the project runs.

7.3 Course Outcomes — Evidence Summary

CO1 — Apply machine learning techniques to a real problem: a nine-model benchmark on an identical stratified split, reported in Table 6.1 and Figure 6.2, with the winner promoted on measured evidence.

CO2 — Design and engineer features from domain understanding: 29 raw attributes expanded into 79 criminologically-motivated features (Section 4.3), with the contribution isolated by a controlled ablation (+3.55 points).

CO3 — Evaluate and interpret model performance critically: the full metric suite in Section 6.1, the confusion-matrix error analysis in Section 6.3, and the candid limitations in Section 6.4.

CO4 — Work effectively in a team through an iterative process: the mentor-reviewed weekly log in Table 3.1 records two complete build-test-learn cycles and the specific feedback that redirected Iteration 2.

CO5 — Communicate technical work clearly: this report, the generated project poster, the voice-over presentation, and the self-documenting REST API at /docs.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project developed RiskRadar, an explainable machine learning system that classifies an urban area-hour as Low, Medium or High risk for women's safety from 29 routinely measurable crime and infrastructure attributes. A tuned LightGBM classifier operating on 79 engineered features achieved 98.27% accuracy, 97.68% macro F1 and 0.9993 ROC-AUC on 20,000 unseen records, cross-validated at $98.00\% \pm 0.08$, improving on the project's own earlier baseline of 94.83% and reducing its error rate by 66%. Across all 20,000 hold-out predictions the model produced zero Low-to-High confusions.

Both halves of the driving question were addressed. Risk can be predicted reliably from the available indicators; and the feature-importance profile shows that infrastructure a municipality can change — camera coverage per incident, distance from police, working street lighting — carries substantial predictive weight, with seven of the ten strongest signals being engineered rather than raw. Every objective set out in Section 1.3 was met, and the finding the team did not anticipate is the most useful one: the representation of the data mattered considerably more than the choice of model.

8.2 Future Scope

- Refit and revalidate on audited municipal crime data, and treat every metric in this report as provisional until that is done.
- Introduce spatial cross-validation by holding out entire districts, to test whether the model transfers geographically rather than only across randomly split records.
- Add temporal drift monitoring and a scheduled retraining cycle, so a deployed model does not silently decay as the city changes.
- Replace the three-class target with an ordinal-aware loss function, since the classes are ordered and a Low-to-High error is far more costly than a Low-to-Medium one.
- Apply conformal prediction to produce distribution-free confidence intervals rather than raw probability estimates.
- Extend to route-level risk aggregation and a mobile client, so that the system can advise on a whole journey rather than a single location.

REFERENCES

- [1] L. E. Cohen and M. Felson, "Social change and crime rate trends: A routine activity approach," *American Sociological Review*, vol. 44, no. 4, pp. 588–608, 1979.
- [2] J. Jacobs, *The Death and Life of Great American Cities*. New York, NY, USA: Random House, 1961.
- [3] O. Kounadi, A. Ristea, A. Araujo, and M. Leitner, "A systematic review on spatial crime forecasting," *Crime Science*, vol. 9, no. 7, 2020.
- [4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [6] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems* 30, 2017, pp. 3146–3154.
- [7] E. Catlett, E. Cesario, D. Talia, and A. Vinci, "Spatio-temporal crime predictions in smart cities: A data-driven approach and experiments," *Pervasive and Mobile Computing*, vol. 53, pp. 62–74, 2019.
- [8] A. Bogomolov, B. Lepri, J. Staiano, N. Oliver, F. Pianesi, and A. Pentland, "Once upon a crime: Towards crime prediction from demographics and mobile data," in *Proc. 16th ACM Int. Conf. Multimodal Interaction*, 2014, pp. 427–434.
- [9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems* 30, 2017, pp. 4765–4774.
- [10] M. Mitchell et al., "Model cards for model reporting," in *Proc. Conf. Fairness, Accountability, and Transparency (FAT*)*, 2019, pp. 220–229.
- [11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [12] RiskRadar dataset, "Urban women's safety risk assessment dataset (Chennai metropolitan region)," 100,000 area-hour records, 2026. [Dataset supplied as part of the Machine Learning course; see Appendix A.1.]

APPENDIX

A.1 Full source code and trained artefacts: <https://github.com/venkat200709>

A.2 Complete weekly PBL log and mentor sign-offs. The condensed log appears as Table 3.1; attach the signed weekly review sheets here.

A.3 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
N. Venkatesan	«%»	«%»	«Summarise this member's main contribution»
Neethivendhan T.	«%»	«%»	«Summarise this member's main contribution»